

VLSI Internship – Task 2 Report

Introduction to Verilog HDL and RTL Design of Basic Combinational Circuits

Submitted By

Name: Yuvaraj Naik

Branch: Electronics and Communication Engineering (ECE)

1. Introduction

Verilog HDL (Hardware Description Language) is a programming language used to model, design, and verify digital electronic circuits. It allows designers to describe hardware at different abstraction levels, including behavioral, dataflow, and structural modeling.

RTL (Register Transfer Level) design is a method of describing digital circuits in terms of data flow between registers and logical operations performed on the data. RTL is widely used in VLSI design because it serves as the input for synthesis tools that generate gate-level implementations.

2. Objectives

The objectives of this task are:

- To understand the fundamentals of Verilog HDL.
 - To learn RTL design methodology.
 - To implement basic logic gates using Verilog.
 - To design Half Adder and Full Adder circuits.
 - To verify the functionality through simulation waveforms.
-

3. Software Used

- EDA Playground
 - Icarus Verilog Simulator
 - EPWave Viewer
-

4. Verilog Implementation of Logic Gates

Logic Gates Implemented

- AND Gate
- OR Gate
- NOT Gate

- NAND Gate
- NOR Gate
- XOR Gate

Verilog Code

```
module logic_gates(  
input A,  
input B,  
output AND_OUT,  
output OR_OUT,  
output NOT_A,  
output NAND_OUT,  
output NOR_OUT,  
output XOR_OUT  
);  
  
assign AND_OUT=A&B;  
assign OR_OUT=A|B;  
assign NOT_A=~A;  
assign NAND_OUT=~(A&B);  
assign NOR_OUT=~(A|B);  
assign XOR_OUT=A^B;  
  
endmodule
```

Truth Tables

AND Gate

A B Output

0 0 0

0 1 0

1 0 0

1 1 1

OR Gate

A B Output

0 0 0

0 1 1

1 0 1

1 1 1

NOT Gate

A Output

0 1

1 0

NAND Gate

A B Output

0 0 1

0 1 1

1 0 1

1 1 0

NOR Gate

A B Output

0 0 1

0 1 0

1 0 0

1 1 0

XOR Gate

A B Output

0 0 0

0 1 1

1 0 1

1 1 0

Logic Gate Simulation



Figure 1: Logic Gates Simulation Waveform (Your first screenshot)

Observation

The simulation confirms that all logic gates produce outputs exactly according to their respective truth tables. The generated waveform verifies the correct implementation of AND, OR, NOT, NAND, NOR, and XOR gates.

5. Half Adder

A Half Adder is a combinational circuit used to add two one-bit binary numbers.

Inputs

- A
- B

Outputs

- Sum
- Carry

Logic Equations

Sum = A XOR B

Carry = A AND B

Verilog Code

```
module half_adder(  
    input A,  
    input B,  
    output Sum,  
    output Carry  
);
```

```
    assign Sum=A^B;  
    assign Carry=A&B;
```

```
endmodule
```

Truth Table

A B Sum Carry

0 0 0 0

0 1 1 0

1 0 1 0

1 1 0 1

Half Adder Simulation



Figure 2: Half Adder Simulation Waveform

Observation

The waveform confirms that the XOR gate correctly generates the Sum output while the AND gate generates the Carry output. The outputs match the theoretical Half Adder truth table for all possible input combinations.

6. Full Adder

A Full Adder is a combinational circuit that adds three one-bit binary inputs (A, B, and Carry-In).

Inputs

- A
- B
- Cin

Outputs

- Sum
 - Cout
-

Logic Equations

$$\text{Sum} = A \text{ XOR } B \text{ XOR } \text{Cin}$$

$$\text{Carry} = AB + \text{BCin} + \text{ACin}$$

Verilog Code

```
module full_adder(
input A,
input B,
input Cin,
output Sum,
output Cout
);

assign Sum=A^B^Cin;
assign Cout=(A&B)|(B&Cin)|(A&Cin);

endmodule
```

Truth Table

A B Cin Sum Cout

0 0 0 0 0

0 0 1 1 0

0 1 0 1 0

0 1 1 0 1

1 0 0 1 0

1 0 1 0 1

1 1 0 0 1

1 1 1 1 1

Full Adder Simulation

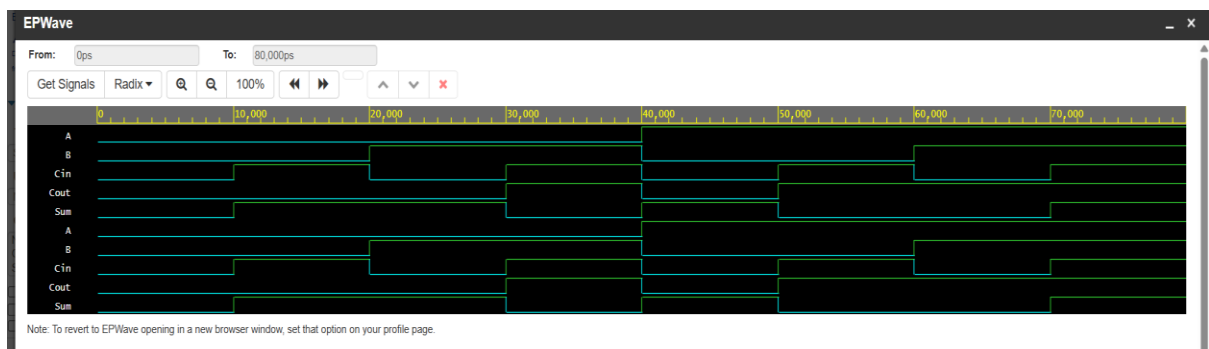


Figure 3: Full Adder Simulation Waveform

Observation

The simulation waveform verifies the Full Adder operation. The Sum output changes according to the XOR operation of all three inputs, while the Carry-Out is generated whenever at least two inputs are HIGH.

Testbench Verification

The Verilog modules were verified using dedicated testbenches. Each possible input combination was applied sequentially, and the resulting waveforms were observed in EPWave. The simulation confirmed that the generated outputs matched the expected truth tables.

Results

- Successfully implemented all six basic logic gates in Verilog HDL.
 - Successfully designed and verified the Half Adder.
 - Successfully designed and verified the Full Adder.
 - Waveform outputs matched the expected theoretical results.
 - Simulation was completed using EDA Playground and EPWave.
-

Conclusion

In this task, Verilog HDL was used to implement and verify basic combinational circuits. The logic gates, Half Adder, and Full Adder were successfully designed using RTL coding techniques. Simulation waveforms confirmed that all circuits behaved according to their truth tables. This task provided practical knowledge of Verilog programming, RTL design methodology, and digital circuit verification, which are essential skills in VLSI design.

References

1. Verilog HDL by Samir Palnitkar.
2. Fundamentals of Digital Logic by Morris Mano.
3. EDA Playground – <https://edaplayground.com>
4. IEEE Verilog HDL Standard.